utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

KernelMind Assignment 4 Report
The Direct Neural Scheduler

SoC 2026 KernelMind Module

July 23, 2026

1 Introduction

This report presents the implementation and evaluation of a Direct Neural Scheduler using Deep Reinforcement Learning. In contrast to high level meta scheduling, the direct control agent observes process attributes in the ready queue and makes tick level process dispatching decisions. We detail the neural architecture, reward design, stability analysis, and final benchmark comparisons against heuristic policies.

2 Answers to Design Questions

2.1 Design Question 1: The Cost of Direct Control

Direct per tick scheduling expands the effective decision horizon compared to heuristic selection. Under meta scheduling, choosing a policy executes multiple time steps per action. Under direct control, an action is selected at every single tick T.

If the agent makes an suboptimal selection early in an episode, credit assignment requires propagating blame backward across long action sequences. The temporal difference target updates via:

$$Q(s_t, a_t) \leftarrow r_t + \gamma Q(s_{t+1}, a_{t+1}) \tag{1}$$

Because the discount factor $\gamma < 1$ scales reward signals exponentially over T steps, credit propagation across 50 to 100 per tick transitions experiences higher variance and slower gradient convergence than meta scheduling.

2.2 Design Question 2: Reward Shaping and Starvation Prevention

To prevent long job starvation, our reward balances CPU throughput against execution balance. Rather than penalizing raw wait time, which causes an agent to abandon long jobs because their long execution time accumulates queue wait penalties, we use a virtual runtime variance penalty.

Let v_i denote the virtual runtime of process i in the ready queue, and $\bar{v}$ denote the mean virtual runtime. The variance term is defined as:

$$\mathrm{Var}(v) = \frac{1}{N}\sum_{i=1}^{N}(v_i - \bar{v})^2 \tag{2}$$

By penalizing $\sqrt{\mathrm{Var}(v)}$, the agent is incentivized to distribute execution ticks across all active processes. Running a starved process directly reduces variance without inheriting compounding wait time penalties during its multi tick execution.

2.3 Design Question 3: Diagnosing Instability

To ensure stability across independent random seeds, two primary metrics were logged during training:

1. **Mean Q Value Magnitude**: We tracked Q value plateaus to verify convergence toward the theoretical limit $R_{\max}/(1 \setminus \gamma)$. Unbounded growth indicates Bellman divergence.

2. **Signed Mean Temporal Difference Error**: We monitored $\mathbb{E}[y_t - Q(s_t, a_t)]$. Symmetric fluctuations around zero confirm stable value approximation. Target network synchronization every 500 steps eliminated moving target oscillations.

2.4 Design Question 4: Output Analysis

The direct neural scheduler successfully learned a policy positioning itself on the Pareto frontier between Shortest Job First and Round Robin. Shortest Job First achieves optimal Mean Wait Time but sacrifices fairness by starving longer jobs. Round Robin achieves near unified fairness at the expense of higher mean wait times. The neural scheduler dynamically trades off a minor increase in mean wait time to achieve a substantially higher Jain Fairness Index.

3 Comparative Performance Results

Table 1 summarizes the benchmark evaluation over identical test workloads comparing First Come First Served, Shortest Job First, Round Robin, Random selection, and our trained Direct Neural Scheduler.

4 Architectural Insights and Masking

The Direct Scheduler Network utilizes multihead self attention to evaluate candidate processes independently of queue slot positions. Two distinct masking mechanisms are strictly necessary:

Table 1: System Performance Metrics Comparison

Policy	**Mean Wait Time**	**P90 Wait Time**	**Jain Fairness Index**
First Come First Served	42.85	88.20	0.741
Shortest Job First	18.40	51.10	0.682
Round Robin (q = 4)	34.12	69.40	0.915
Random Selection	55.30	112.50	0.580
Direct Neural Scheduler	**22.65**	**54.30**	**0.884**

1. **Attention Key Padding Mask**: Prevents zero padded sequence positions from contributing key and value vectors during attention score calculation.

2. **Action Output Masking**: Sets Q values of invalid padding slots to negative infinity prior to action selection, preventing the agent from selecting non existent processes.

5 Conclusion

The Direct Neural Scheduler demonstrates that deep reinforcement learning can learn effective CPU scheduling strategies directly from raw process state representations without relying on fixed heuristics.